

STM Creator API Manual

This manual documents the STM Creator backend API as represented by the current backend and frontend source structure. It covers base URLs, authentication, roles, errors, rate limits, REST endpoints, Socket.IO events, and curl examples.

Production deployment:

- Frontend editor: `https://stm-8nizc.ondigitalocean.app/editor`
- Backend API used by the frontend source: `https://hammerhead-app-t8l9y.ondigitalocean.app`

Local defaults:

- Frontend: `http://localhost:5173`
- Backend: `http://localhost:3000`

1. Base Information

1.1 Base URL

For local development:

```
http://localhost:3000
```

For the current deployed backend:

```
https://hammerhead-app-t8l9y.ondigitalocean.app
```

The deployed frontend editor is:

```
https://stm-8nizc.ondigitalocean.app/editor
```

1.2 Content Type

Most REST requests and responses use JSON:

```
Content-Type: application/json  
Accept: application/json
```

File import/export is primarily handled by the frontend. The backend model save endpoint receives normalized JSON model payloads.

1.3 Authentication Header

Authenticated REST calls use:

```
Authorization: Bearer <jwt>
```

Socket.IO uses the same JWT concept during the connection handshake.

2. Authentication

The backend uses JWT authentication. Login returns a token. The frontend stores and reuses that token for REST calls and collaboration socket connections.

General rules:

- Public endpoints include health checks, signup, verification, resend verification, and login.
- Most model, collaboration, notification, and admin endpoints require authentication.
- Some endpoints require verified email.
- Some endpoints require global `Admin`.
- Model-specific endpoints can require `viewer`, `editor`, or `owner`.

Example:

```
GET /models/all HTTP/1.1
Host: localhost:3000
Authorization: Bearer eyJ...
```

3. Roles and Permissions

3.1 Global Roles

Role	Meaning
<code>Admin</code>	System administrator. Can access admin APIs and high-privilege model operations.
<code>Editor</code>	Normal editing user. Can create and edit where model permission allows it.
<code>Viewer</code>	Read-only default account role.

3.2 Model Roles

Role	Meaning
<code>owner</code>	Full control of a model, including permission management.
<code>editor</code>	Can edit and save a model.
<code>viewer</code>	Can view a model.

Backend permission checks are authoritative. Frontend permission checks are for user experience only.

4. Error Format

The backend uses centralized error handling. Exact fields can vary by error source, but the normal response envelope is conceptually:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Request validation failed",
    "details": []
  },
  "requestId": "req_..."
}
```

Common HTTP status codes:

Status	Meaning
200	Success.
201	Created.
204	Success with no response body.
400	Invalid request or validation failure.
401	Missing, invalid, or expired authentication.
403	Authenticated but not permitted.
404	Resource not found.
409	Conflict, such as duplicate data or active lock.
422	Semantically invalid payload where used.
429	Rate limit exceeded.
500	Server error.

Common error codes:

Code	Meaning
VALIDATION_ERROR	Request body, query, or params did not match schema.
AUTH_REQUIRED	Missing authentication.
INVALID_TOKEN	JWT could not be verified.
EMAIL_NOT_VERIFIED	User must verify email before using the endpoint.
FORBIDDEN	User lacks required role or permission.
NOT_FOUND	Resource does not exist or is not visible to the user.
CONFLICT	Operation conflicts with existing state.
LOCKED	Model or entity is locked.

5. Rate Limits

The backend includes in-memory rate limiting middleware. Authentication and verification endpoints are likely to be more tightly limited than normal reads.

Operational notes:

- In-memory rate limits reset when the backend process restarts.
- In multi-instance production, in-memory limits are per instance unless replaced by shared storage.
- A 429 response means the client should wait before retrying.

6. Auth API

GET /auth/health

Health check for the auth route.

Auth: public.

Response example:

```
{
  "status": "ok"
}
```

POST /auth/signup

Creates a user and sends or prepares email verification.

Auth: public.

Request example:

```
{
  "name": "Ada Lovelace",
  "email": "ada@example.com",
  "password": "CorrectHorseBatteryStaple123!"
}
```

Response example:

```
{
  "message": "Signup successful. Please verify your email."
}
```

Notes:

- New users are typically created with global role `Viewer`.
- Duplicate email should return a conflict or validation-style error.
- Password is never returned.

POST /auth/verify

Verifies a user's email address using a token.

Auth: public.

Request example:

```
{
  "token": "verification-token"
}
```

Response example:

```
{
  "message": "Email verified"
}
```

POST /auth/resend-verification

Requests another verification email.

Auth: public.

Request example:

```
{
  "email": "ada@example.com"
}
```

Response example:

```
{
  "message": "Verification email sent"
}
```

POST /auth/login

Authenticates a user and returns a JWT.

Auth: public.

Request example:

```
{
  "email": "ada@example.com",
  "password": "CorrectHorseBatteryStaple123!"
}
```

Response example:

```
{
  "token": "eyJ...",
  "user": {
    "id": 1,
    "name": "Ada Lovelace",
    "email": "ada@example.com",
    "role": "Editor",
    "emailVerified": true
  }
}
```

Common failures:

- Invalid credentials.
- Email not verified.
- Rate limit exceeded.

7. Models API

GET /models/health

Health check for the models route.

Auth: public or lightweight depending on middleware order.

Response example:

```
{
  "status": "ok"
}
```

GET /models/all

Returns models visible to the current user. Admin users may see more than normal users.

Auth: required.

Response example:

```
[
  {
    "name": "Example STM",
    "description": "Example model",
    "is_template": false
  }
]
```

GET /models/assigned

Returns models assigned or shared with the current user.

Auth: required.

Response example:

```
[
  {
    "name": "Shared STM",
    "role": "editor"
  }
]
```

GET /models/templates

Returns available template models.

Auth: required.

Response example:

```
[
  {
    "name": "Woodlands",
    "is_template": true
  },
  {
    "name": "Grasslands",
    "is_template": true
  },
  {
    "name": "Shrublands",
    "is_template": true
  }
]
```

GET /models/:name

Returns a complete model by name.

Auth: required.

Required model role: usually `viewer` or higher.

Path params:

Param	Description
<code>name</code>	Model name. URL-encode if it contains spaces or special characters.

Response shape:

```
{
  "name": "Example STM",
  "description": "Example model",
  "is_template": false,
  "states": [
    {
      "id": "state-1",
      "name": "State 1",
      "description": "State description",
      "position": {
        "x": 100,
        "y": 200
      }
    }
  ],
  "transitions": [
    {
      "id": "transition-1",
      "source": "state-1",
      "target": "state-2",
      "description": "Transition description"
    }
  ]
}
```

Exact field names depend on the backend model service and frontend conversion utilities. Treat the current TypeScript types as the source of truth during development.

POST /models/save

Creates or updates a model.

Auth: required.

Required permission:

- Create: global `Editor` or `Admin`, depending on current policy.
- Update: model `editor` or `owner`, or `Admin`.

Request example:

```
{
  "name": "Example STM",
  "description": "Example model",
  "is_template": false,
  "states": [
    {
      "id": "state-1",
      "name": "State 1",
      "description": "State description",
      "vastClass": "A",
      "position": {
        "x": 100,
        "y": 200
      }
    }
  ],
  "transitions": [
    {
      "id": "transition-1",
      "source": "state-1",
      "target": "state-2",
      "driver": "Fire",
      "precondition": "Dry season",
      "description": "Fire shifts the system to State 2"
    }
  ]
}
```

Response example:

```
{
  "message": "Model saved",
  "name": "Example STM"
}
```

Common failures:

- Missing authentication.
- Email not verified.
- Insufficient global role.
- Insufficient model role.
- Review lock is active.
- Validation error.
- Database constraint violation.

POST /models/from-template/:name

Clones a template model into a user-editable model.

Auth: required.

Required permission: user must be allowed to create models.

Path params:

Param	Description
name	Template name, such as Woodlands , Grasslands , or Shrublands .

Request example:

```
{
  "newName": "My Woodlands STM"
}
```

Response example:

```
{
  "message": "Model created from template",
  "name": "My Woodlands STM"
}
```

DELETE /models/:name

Deletes a model.

Auth: required.

Required permission: owner/admin policy.

Response example:

```
{
  "message": "Model deleted"
}
```

DELETE /models/:name/states/:stateId

Deletes a state from a model.

Auth: required.

Required model role: editor or owner .

Path params:

Param	Description
name	Model name.
stateId	State business ID.

Response example:

```
{
  "message": "State deleted"
}
```

DELETE /models/:name/transitions/:transitionId

Deletes a transition from a model.

Auth: required.

Required model role: `editor` or `owner`.

Path params:

Param	Description
<code>name</code>	Model name.
<code>transitionId</code>	Transition business ID.

Response example:

```
{
  "message": "Transition deleted"
}
```

GET /models/:name/review-lock

Returns review-lock status for a model.

Auth: required.

Required model role: usually `viewer` or higher.

Response example:

```
{
  "locked": true,
  "reason": "Under review",
  "lockedBy": {
    "id": 1,
    "email": "admin@example.com"
  },
  "lockedAt": "2026-05-15T00:00:00.000Z"
}
```

POST /models/:name/review-lock

Applies a review lock.

Auth: required.

Required role: `Admin`.

Request example:

```
{
  "reason": "Final review before publication"
}
```

Response example:

```
{
  "message": "Review lock applied"
}
```

DELETE `/models/:name/review-lock`

Removes a review lock.

Auth: required.

Required role: `Admin`.

Response example:

```
{
  "message": "Review lock removed"
}
```

8. Model Permissions API

Mounted under:

```
/models/:name/permissions
```

GET `/models/:name/permissions`

Lists model permissions.

Auth: required.

Required permission: owner/admin policy.

Response example:

```
[
  {
    "email": "ada@example.com",
    "role": "owner",
    "userId": 1
  },
  {
    "email": "grace@example.com",
    "role": "editor",
    "userId": 2
  }
]
```

Implementation note:

- Some frontend code may expect an object such as `{ "permissions": [...] }`.
- Confirm frontend and backend response shape when maintaining the sharing modal.

PUT /models/:name/permissions/:email

Grants or updates a user's model role.

Auth: required.

Required permission: owner/admin policy.

Path params:

Param	Description
name	Model name.
email	Target user's email address. URL-encode it.

Request example:

```
{
  "role": "editor"
}
```

Response example:

```
{
  "message": "Permission updated",
  "permission": {
    "email": "grace@example.com",
    "role": "editor"
  }
}
```

DELETE /models/:name/permissions/:email

Removes a user's model permission.

Auth: required.

Required permission: owner/admin policy.

Response example:

```
{
  "message": "Permission removed"
}
```

9. REST Locks API

REST locks are model-level locks. They are separate from Socket.IO entity locks.

POST /models/:name/lock/acquire

Acquires a model-level lock.

Auth: required.

Request example:

```
{
  "ttlSeconds": 120,
  "lockType": "edit"
}
```

Response example:

```
{
  "acquired": true,
  "lock": {
    "modelName": "Example STM",
    "lockedBy": "ada@example.com",
    "expiresAt": "2026-05-15T00:02:00.000Z",
    "lockType": "edit"
  }
}
```

Conflict response example:

```
{
  "error": {
    "code": "LOCKED",
    "message": "Model is already locked"
  }
}
```

POST /models/:name/lock/renew

Renews a model-level lock held by the current user.

Auth: required.

Request example:

```
{
  "ttlSeconds": 120
}
```

Response example:

```
{
  "renewed": true
}
```

POST /models/:name/lock/release

Releases a model-level lock held by the current user.

Auth: required.

Response example:

```
{
  "released": true
}
```

GET /models/:name/lock

Returns current model-level lock state.

Auth: required.

Response example:

```
{
  "locked": false,
  "lock": null
}
```

10. Drivers API

GET /drivers/search

Searches transition drivers.

Auth: required.

Query params:

Param	Description
q	Search text.
limit	Optional max results.

Request example:

```
GET /drivers/search?q=fire&limit=10
```

Response example:

```
[
  {
    "id": 1,
    "name": "Fire",
    "description": "Fire-related transition driver"
  }
]
```

GET /drivers/:id/sub-drivers

Lists sub-drivers for a driver.

Auth: required.

Path params:

Param	Description
id	Driver ID.

Response example:

```
[
  {
    "id": 10,
    "name": "High-intensity fire",
    "driverId": 1
  }
]
```

11. Collaboration REST API

GET /collab/:modelName/activity

Returns recent collaboration activity for a model.

Auth: required.

Required model role: **viewer** or higher.

Query params:

Param	Description
limit	Optional max number of activity records.

Response example:

```
[
  {
    "id": 100,
    "type": "comment:create",
    "actorEmail": "ada@example.com",
    "modelName": "Example STM",
    "createdAt": "2026-05-15T00:00:00.000Z",
    "payload": {}
  }
]
```

GET /collab/:modelName/comments

Lists comments for a model.

Auth: required.

Required model role: `viewer` or higher.

Query params:

Param	Description
limit	Optional max comments.
offset	Optional pagination offset.
resolved	Optional resolved filter.

Response example:

```
[
  {
    "id": 1,
    "body": "Please review this transition.",
    "authorEmail": "ada@example.com",
    "resolved": false,
    "createdAt": "2026-05-15T00:00:00.000Z"
  }
]
```

POST /collab/:modelName/comments

Creates a comment.

Auth: required.

Required model role: `viewer` or higher, depending on policy.

Request example:

```
{
  "body": "Please review this transition.",
  "entityType": "transition",
  "entityId": "transition-1",
  "mentions": ["grace@example.com"]
}
```

Response example:

```
{
  "id": 1,
  "body": "Please review this transition.",
  "authorEmail": "ada@example.com",
  "resolved": false
}
```

Side effects:

- Activity record may be created.
- Mention notifications may be created.
- Socket event `comment:new` may be broadcast.

PATCH `/collab/:modelName/comments/:id/resolve`

Marks a comment as resolved or unresolved.

Auth: required.

Request example:

```
{
  "resolved": true
}
```

Response example:

```
{
  "id": 1,
  "resolved": true
}
```

DELETE `/collab/:modelName/comments/:id`

Deletes a comment.

Auth: required.

Required permission depends on policy, usually comment author, editor, owner, or admin.

Response example:

```
{
  "message": "Comment deleted"
}
```

GET /collab/:modelName/milestones

Lists milestones for a model.

Auth: required.

Required model role: `viewer` or higher.

Response example:

```
[
  {
    "id": 1,
    "name": "Before review",
    "description": "Snapshot before final review",
    "createdBy": "ada@example.com",
    "createdAt": "2026-05-15T00:00:00.000Z"
  }
]
```

POST /collab/:modelName/milestones

Creates a milestone snapshot.

Auth: required.

Required model role: `editor` or `owner`.

Request example:

```
{
  "name": "Before review",
  "description": "Snapshot before final review"
}
```

Response example:

```
{
  "id": 1,
  "name": "Before review"
}
```

Side effects:

- Captures current model data.

- Broadcasts `milestone:created` where collaboration is active.

GET `/collab/:modelName/milestones/:id`

Returns a milestone and its stored snapshot.

Auth: required.

Required model role: `viewer` or higher.

Response example:

```
{
  "id": 1,
  "name": "Before review",
  "snapshot": {
    "states": [],
    "transitions": []
  }
}
```

DELETE `/collab/:modelName/milestones/:id`

Deletes a milestone.

Auth: required.

Required model role: `editor` or `owner`, depending on policy.

Response example:

```
{
  "message": "Milestone deleted"
}
```

POST `/collab/:modelName/milestones/:id/restore`

Restores a model from a milestone snapshot.

Auth: required.

Required model role: `editor` or `owner`.

Response example:

```
{
  "message": "Model restored",
  "modelName": "Example STM"
}
```

Side effects:

- Current model data is replaced by snapshot data.
- Socket event `model:restored` may be broadcast.

12. Notifications API

GET /notifications/mentions

Lists mention notifications for the current user.

Auth: required.

Response example:

```
[
  {
    "id": 1,
    "modelName": "Example STM",
    "commentId": 10,
    "read": false,
    "createdAt": "2026-05-15T00:00:00.000Z"
  }
]
```

PATCH /notifications/mentions/read

Marks mention notifications as read.

Auth: required.

Request example:

```
{
  "ids": [1, 2, 3]
}
```

Response example:

```
{
  "updated": 3
}
```

13. Admin API

Admin endpoints are mounted under:

```
/api/admin
```

Auth: required.

Required global role: `Admin`.

GET /api/admin/stats

Returns admin dashboard statistics.

Response example:

```
{
  "users": 12,
  "models": 5,
  "sessions": 8,
  "comments": 20
}
```

GET /api/admin/users

Lists users for admin management.

Query params:

Param	Description
limit	Optional page size.
offset	Optional pagination offset.
q	Optional search text.

Response example:

```
[
  {
    "id": 1,
    "name": "Ada Lovelace",
    "email": "ada@example.com",
    "role": "Editor",
    "emailVerified": true,
    "createdAt": "2026-05-15T00:00:00.000Z"
  }
]
```

PATCH /api/admin/users/:id/role

Updates a user's global role.

Request example:

```
{
  "role": "Editor"
}
```

Response example:

```
{
  "id": 1,
  "role": "Editor"
}
```

DELETE /api/admin/users/:id/sessions

Deletes or invalidates a user's sessions.

Response example:

```
{
  "message": "Sessions deleted"
}
```

DELETE /api/admin/users/:id

Deletes a user where policy allows it.

Response example:

```
{
  "message": "User deleted"
}
```

GET /api/admin/audit-log

Returns audit log entries.

Query params:

Param	Description
limit	Optional page size.
offset	Optional pagination offset.
actor	Optional actor filter.
action	Optional action filter.

Response example:

```
[
  {
    "id": 1,
    "actorEmail": "admin@example.com",
    "action": "user.role.update",
    "target": "ada@example.com",
    "createdAt": "2026-05-15T00:00:00.000Z",
    "metadata": {}
  }
]
```

14. Socket.IO API

Socket.IO powers live collaboration.

14.1 Connection

Client connects to the backend Socket.IO server and supplies the JWT in auth data.

Frontend concept:

```
io(API_BASE_URL, {
  auth: {
    token
  }
});
```

Server authenticates the token and associates the socket with a user.

Connection failures:

- Missing token.
- Invalid token.
- Expired token.
- User not found.
- CORS origin mismatch.

14.2 Room Model

Collaboration is organized by model room.

Typical room key:

```
model:<modelName>
```

The client should join the room for the currently opened model and leave or disconnect when switching models.

14.3 Client to Server Events

room:join

Join a model collaboration room.

Payload:

```
{
  "modelName": "Example STM"
}
```

Expected server behavior:

- Validate access.
- Add socket to model room.
- Send `presence:sync`.
- Broadcast `presence:join`.
- Send recent activity.

cursor:move

Broadcast cursor position.

Payload:

```
{
  "modelName": "Example STM",
  "x": 120,
  "y": 300
}
```

viewport:update

Broadcast viewport state.

Payload:

```
{
  "modelName": "Example STM",
  "x": 0,
  "y": 0,
  "zoom": 1.2
}
```

lock:acquire

Acquire an entity lock.

Payload:

```
{
  "modelName": "Example STM",
  "entityType": "state",
  "entityId": "state-1"
}
```

Server may respond with `lock:acquired` or `lock:denied` .

`lock:release`

Release an entity lock.

Payload:

```
{
  "modelName": "Example STM",
  "entityType": "state",
  "entityId": "state-1"
}
```

`entity:patch`

Broadcast a collaborative patch for an entity.

Payload example:

```
{
  "modelName": "Example STM",
  "entityType": "state",
  "entityId": "state-1",
  "patch": {
    "name": "Updated state name"
  }
}
```

Server should validate permissions and lock ownership where required.

14.4 Server to Client Events

`presence:sync`

Initial room presence payload.

Payload example:

```
{
  "users": [
    {
      "id": 1,
      "email": "ada@example.com",
      "name": "Ada Lovelace"
    }
  ]
}
```

presence:join

Broadcast when a user joins the room.

Payload example:

```
{
  "user": {
    "id": 2,
    "email": "grace@example.com",
    "name": "Grace Hopper"
  }
}
```

presence:leave

Broadcast when a user leaves the room.

Payload example:

```
{
  "userId": 2,
  "email": "grace@example.com"
}
```

activity:recent

Sends recent activity after joining a room.

Payload example:

```
{
  "items": [
    {
      "type": "comment:create",
      "actorEmail": "ada@example.com",
      "createdAt": "2026-05-15T00:00:00.000Z"
    }
  ]
}
```

activity:new

Broadcasts a new activity item.

Payload example:

```
{
  "type": "milestone:create",
  "actorEmail": "ada@example.com",
  "createdAt": "2026-05-15T00:00:00.000Z",
  "payload": {}
}
```

cursor:move

Broadcasts another user's cursor.

Payload example:

```
{
  "userId": 2,
  "email": "grace@example.com",
  "x": 120,
  "y": 300
}
```

viewport:update

Broadcasts another user's viewport.

Payload example:

```
{
  "userId": 2,
  "x": 0,
  "y": 0,
  "zoom": 1.2
}
```

lock:acquired

Confirms or broadcasts an acquired entity lock.

Payload example:

```
{
  "entityType": "state",
  "entityId": "state-1",
  "lockedBy": {
    "id": 1,
    "email": "ada@example.com"
  },
  "expiresAt": "2026-05-15T00:02:00.000Z"
}
```

lock:denied

Indicates a lock could not be acquired.

Payload example:

```
{
  "entityType": "state",
  "entityId": "state-1",
  "lockedBy": {
    "id": 2,
    "email": "grace@example.com"
  },
  "reason": "Entity is already locked"
}
```

lock:released

Broadcasts lock release.

Payload example:

```
{
  "entityType": "state",
  "entityId": "state-1"
}
```

entity:patch

Broadcasts a remote entity patch.

Payload example:

```
{
  "entityType": "state",
  "entityId": "state-1",
  "patch": {
    "name": "Updated state name"
  },
  "updatedBy": {
    "id": 1,
    "email": "ada@example.com"
  }
}
```

comment:new

Broadcasts a newly created comment.

Payload example:

```
{
  "comment": {
    "id": 1,
    "body": "Please review this transition.",
    "authorEmail": "ada@example.com"
  }
}
```

milestone:created

Broadcasts a newly created milestone.

Payload example:

```
{
  "milestone": {
    "id": 1,
    "name": "Before review"
  }
}
```

model:restored

Broadcasts that the model was restored from a milestone.

Payload example:

```
{
  "modelName": "Example STM",
  "milestoneId": 1
}
```

error:validation

Sent when a socket payload fails validation.

Payload example:

```
{
  "message": "Invalid payload",
  "details": []
}
```

error:lock

Sent when a lock operation fails.

Payload example:

```
{
  "message": "Entity is already locked",
  "entityType": "state",
  "entityId": "state-1"
}
```

error:patch

Sent when an entity patch fails.

Payload example:

```
{
  "message": "Patch rejected"
}
```

14.5 Disconnect Cleanup

When a socket disconnects, the backend should:

- Remove the user from active room presence.
- Broadcast `presence:leave`.
- Release transient locks held by that socket.
- Leave persistent data such as comments and milestones unchanged.

15. curl Examples

Set a base URL:

```
BASE_URL="http://localhost:3000"
```

For production:

```
BASE_URL="https://hammerhead-app-t819y.ondigitalocean.app"
```

Login

```
curl -s -X POST "$BASE_URL/auth/login" \  
-H "Content-Type: application/json" \  
-d '{  
  "email": "ada@example.com",  
  "password": "CorrectHorseBatteryStaple123!"  
}'
```

Read a Model

```
curl -s "$BASE_URL/models/Example%20STM" \  
-H "Authorization: Bearer $TOKEN"
```

Save a Model

```
curl -s -X POST "$BASE_URL/models/save" \  
-H "Authorization: Bearer $TOKEN" \  
-H "Content-Type: application/json" \  
-d '{  
  "name": "Example STM",  
  "description": "Example model",  
  "states": [],  
  "transitions": []  
}'
```

Search Drivers

```
curl -s "$BASE_URL/drivers/search?q=fire&limit=10" \  
-H "Authorization: Bearer $TOKEN"
```

Create a Comment

```
curl -s -X POST "$BASE_URL/collab/Example%20STM/comments" \  
-H "Authorization: Bearer $TOKEN" \  
-H "Content-Type: application/json" \  
-d '{  
  "body": "Please review this transition.",  
  "entityType": "transition",  
  "entityId": "transition-1",  
  "mentions": ["grace@example.com"]  
}'
```

Create a Milestone


```
curl -s -X POST "$BASE_URL/collab/Example%20STM/milestones" \  
-H "Authorization: Bearer $TOKEN" \  
-H "Content-Type: application/json" \  
-d '{  
  "name": "Before review",  
  "description": "Snapshot before final review"  
}'
```

Grant Model Permission

```
curl -s -X PUT "$BASE_URL/models/Example%20STM/permissions/grace%40example.com" \  
-H "Authorization: Bearer $TOKEN" \  
-H "Content-Type: application/json" \  
-d '{  
  "role": "editor"  
}'
```

Apply Review Lock

```
curl -s -X POST "$BASE_URL/models/Example%20STM/review-lock" \  
-H "Authorization: Bearer $TOKEN" \  
-H "Content-Type: application/json" \  
-d '{  
  "reason": "Final review before publication"  
}'
```

16. API Maintenance Checklist

When changing an API:

1. Update the backend route, validation schema, and service.
2. Update authorization and permission checks.
3. Update unit or integration tests.
4. Update frontend API clients.
5. Update TypeScript types.
6. Update this API manual.
7. Update Swagger/OpenAPI docs where applicable.
8. Update Postman collection if the endpoint is included in test workflows.